In [1]:
```
pip install pydantic
```

```
Requirement already satisfied: pydantic in c:\users\kupit\miniconda3\envs\py311\lib\site-packages (2.12.5)
Requirement already satisfied: annotated-types>=0.6.0 in c:\users\kupit\miniconda3\envs\py311\lib\site-packages (from pydantic) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.5 in c:\users\kupit\miniconda3\envs\py311\lib\site-packages (from pydantic) (2.41.5)
Requirement already satisfied: typing-extensions>=4.14.1 in c:\users\kupit\miniconda3\envs\py311\lib\site-packages (from pydantic) (4.15.0)
Requirement already satisfied: typing-inspection>=0.4.2 in c:\users\kupit\miniconda3\envs\py311\lib\site-packages (from pydantic) (0.4.2)
Note: you may need to restart the kernel to use updated packages.
```

In [8]:
```
"""
API with JSON Validation using Pydantic - Complete Solution
Validate JSON input using Pydantic before processing
"""

from pydantic import BaseModel, Field, validator
from typing import Optional
import json

print("="*50)
print("PYDANTIC BASICS")
print("="*50)

class SimpleProduct(BaseModel):
    """A simple product model for validation."""
    name: str
    price: float
    quantity: int = 1  # Default value

    @validator('price')
    def price_must_be_positive(cls, v):
        """Validate that price is positive."""
        if v <= 0:
            raise ValueError('Price must be positive')
        return v

    @validator('quantity')
    def quantity_must_be_positive(cls, v):
        """Validate that quantity is positive."""
        if v <= 0:
            raise ValueError('Quantity must be positive')
        return v

# Test validation
print("\n1. Valid data:")
try:
    product1 = SimpleProduct(name="Widget", price=10.99, quantity=5)
    print(f"  ✓ Valid: {product1.name} - ${product1.price}")
except Exception as e:
    print(f"  ✗ Error: {e}")

print("\n2. Invalid data (negative price):")
try:
    product2 = SimpleProduct(name="Widget", price=-10.99)
except Exception as e:
    print(f"  ✓ Validation error (expected): {e}")

print("\n✓ Pydantic basics working!")

"""
Complete workflow: Hugging Face dataset -> Pydantic validation -> OpenAI API integration
"""

"""
Workflow demonstration with real Hugging Face dataset:
- Invalid data is rejected before API calls
- Valid data is processed successfully
- Errors are handled gracefully
- OpenAI API generates e-commerce listings using the provided prompt template
"""

"""
Lab demonstration with dynamic results:
- Real dataset validation
- Show invalid products only if they exist
- Generate OpenAI JSON listings for valid products
- Save to file
- Handle errors gracefully
"""

"""
Lab demonstration:
- Validate real fields from dataset
- Show invalid products (based on real fields)
- Generate OpenAI JSON listings for valid products
- Save to file
- Handle errors gracefully
"""

"""
Lab demonstration:
- Validate real fields from dataset (gender, category, subCategory, articleType, baseColour, season, usage, productDisplayName)
- Show invalid products (based on real fields)
- Generate OpenAI JSON listings for valid products
- Save to file
- Handle errors gracefully
"""

"""
Full dataset validation + OpenAI listing generation demo
- Validate real fields from dataset
- Show invalid products
- Generate OpenAI JSON listings for first 5 valid products
- Save to file
- Handle errors gracefully
"""

"""
Full dataset validation + OpenAI listing generation demo with standardized output
- Validate real fields from dataset
- Show invalid products
- Generate OpenAI JSON listings for first 5 valid products
- Validate API output with Pydantic
- Save to file
- Handle errors gracefully
"""

from datasets import load_dataset
from pydantic import BaseModel, Field, field_validator, ValidationError
from typing import Optional, List
import os
from dotenv import load_dotenv
import openai
import json

# ========================================
# STEP 0: Load API key from .env
# ========================================
load_dotenv()
openai.api_key = os.getenv("OPENAI_API_KEY")

# ========================================
# STEP 1: Pydantic Product Model
# ========================================
class Product(BaseModel):
    gender: str
    master_category: str
    sub_category: str
    article_type: str
    base_colour: str
    season: str
    usage: str
    product_name: str  # productDisplayName
    price: float = Field(10.0, description="Price, default 10")
    quantity: int = Field(1, description="Quantity, default 1")

    @field_validator('price')
    def price_must_be_positive(cls, v):
        if v <= 0:
            raise ValueError('Price must be positive')
        return v

    @field_validator('quantity')
    def quantity_must_be_positive(cls, v):
        if v <= 0:
            raise ValueError('Quantity must be positive')
        return v

    @field_validator(
        "gender", "master_category", "sub_category", "article_type",
        "base_colour", "season", "usage", "product_name"
    )
    def must_not_be_empty(cls, v):
        if not v or not v.strip():
            raise ValueError('Field must not be empty')
        return v

# ========================================
# STEP 2: Pydantic Model for Product Listing (API output)
# ========================================
class ProductListing(BaseModel):
    title: str
    description: str
    features: List[str]
    keywords: str

# ========================================
# STEP 3: Load Hugging Face Dataset (FULL)
# ========================================
dataset = load_dataset("ashraq/fashion-product-images-small", split="train")  # full dataset
print(f"✓ Loaded {len(dataset)} products")

# ========================================
# STEP 4: Transform Dataset for Pydantic
# ========================================
products_list = []
for item in dataset:
    products_list.append({
        "gender": item.get("gender", ""),.strip(),
        "master_category": item.get("masterCategory", "").strip(),
        "sub_category": item.get("subCategory", "").strip(),
        "article_type": item.get("articleType", "").strip(),
        "base_colour": item.get("baseColour", "").strip(),
        "season": item.get("season", "").strip(),
        "usage": item.get("usage", "").strip(),
        "product_name": item.get("productDisplayName", "").strip(),
        "price": 10.0,      # default
        "quantity": 1       # default
    })

# ========================================
# STEP 5: Validate Products
# ========================================
validated_products = []
invalid_products = []

for idx, item in enumerate(products_list):
    try:
        product = Product(**item)
        validated_products.append(product)
    except ValidationError as e:
        invalid_products.append({"index": idx, "data": item, "error": str(e)})

# ========================================
# STEP 6: Report Validation Results
# ========================================
print("\n=== Validation Summary ===")
print(f"Total products: {len(products_list):,}")
print(f"✅ Valid products: {len(validated_products):,}")
print(f"❌ Invalid products: {len(invalid_products):,}")

if invalid_products:
    print("\nFirst 5 invalid products:")
    for p in invalid_products[:5]:
        print(f"  Index {p['index']}: {p['data']}")
        print(f"  Error: {p['error']}\n")

# ========================================
# STEP 7: Prompt Template
# ========================================
def create_product_listing_prompt(product: Product):
    prompt = f"""You are an expert e-commerce copywriter. Analyze the product image and create a compelling product listing.

Product Information:
- Name: {product.product_name}
- Gender: {product.gender}
- Master Category: {product.master_category}
- Sub Category: {product.sub_category}
- Article Type: {product.article_type}
- Color: {product.base_colour}
- Season: {product.season}
- Usage: {product.usage}
- Price: ${product.price:.2f}

Please create a professional product listing that includes:

1. **Product Title** (catchy, SEO-friendly, 60 characters max)
2. **Product Description** (detailed, 150-200 words)
   - Highlight key features and benefits
   - Use persuasive language
   - Include all relevant attributes (gender, color, season, usage)
3. **Key Features** (bullet points, 5-7 items)
4. **SEO Keywords** (comma-separated, 10-15 relevant keywords)

Format your response as JSON with the following structure:
{{
  "title": "Product title here",
  "description": "Full description here",
  "features": ["Feature 1", "Feature 2", ...],
  "keywords": "keyword1, keyword2, ..."
}}

Be specific about what you see in the image. Mention colors, materials, design elements, seasonality, gender, usage, and any distinctive features."""
    return prompt

# ========================================
# STEP 8: Generate Product Listings via OpenAI + Validate Output
# ========================================
def generate_product_listing(product: Product, model: str = "gpt-4") -> dict:
    prompt = create_product_listing_prompt(product)
    try:
        response = openai.chat.completions.create(
            model=model,
            messages=[{"role": "user", "content": prompt}],
            temperature=0.7,
            max_tokens=400
        )
        content = response.choices[0].message.content.strip()
        try:
            listing_dict = json.loads(content)
        except json.JSONDecodeError:
            print(f"⚠️ Failed to parse JSON for {product.product_name}. Returning raw text.")
            return {"raw_output": content}

        # Validate JSON output via Pydantic
        try:
            listing_obj = ProductListing(**listing_dict)
            return listing_obj.dict()  # standardized format
        except ValidationError as e:
            print(f"⚠️ Listing format invalid for {product.product_name}: {e}")
            return {"raw_output": listing_dict, "error": str(e)}

    except Exception as e:
        print(f"❌ OpenAI API error for {product.product_name}: {e}")
        return {"error": str(e)}

def process_valid_products(products: List[Product]) -> List[dict]:
    processed_results = []
    for product in products[:5]:  # demo: first 5 valid products
        listing = generate_product_listing(product)
        processed_results.append(listing)
    return processed_results

# ========================================
# STEP 9: Process and Save Valid Listings
# ========================================
print("\n=== Generating Listings for First 5 Valid Products ===")
valid_product_listings = process_valid_products(validated_products)

output_file = "valid_product_listings.json"
with open(output_file, "w", encoding="utf-8") as f:
    json.dump(valid_product_listings, f, ensure_ascii=False, indent=2)

# ========================================
# STEP 10: Demo Summary
# ========================================
print("\n=== Demo Summary ===")
if invalid_products:
    print(f"❌ Invalid products found: {len(invalid_products)} (not sent to API)")
else:
    print("✅ No invalid products found, all products are valid")

print(f"✅ {len(valid_product_listings)} valid products processed successfully and JSON listings generated")
print(f"Check file: {output_file}")
print("⚠️ Any API or JSON parsing errors were printed above")
```

```
==================================================
PYDANTIC BASICS
==================================================


1. Valid data:
  ✓ Valid: Widget - $10.99

2. Invalid data (negative price):
  ✗ Validation error (expected): 1 validation error for SimpleProduct
price
  Value error, Price must be positive [type=value_error, input_value=-10.99, input_type=float]
    For further information visit https://errors.pydantic.dev/2.12/v/value_error

✓ Pydantic basics working!
C:\Users\kupit\AppData\Local\Temp\ipykernel_30600\952890231.py:20: PydanticDeprecatedSince20: Pydantic V1 style `@validator` validators are deprecated. You should migrate to Pydantic V2 style `@field_validator` validators, see the migration guide for more details. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
  @validator('price')
C:\Users\kupit\AppData\Local\Temp\ipykernel_30600\952890231.py:27: PydanticDeprecatedSince20: Pydantic V1 style `@validator` validators are deprecated. You should migrate to Pydantic V2 style `@field_validator` validators, see the migration guide for more details. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
  @validator('quantity')
✓ Loaded 44072 products

=== Validation Summary ===
Total products: 44072
✅ Valid products: 44072
❌ Invalid products: 0

=== Generating Listings for First 5 Valid Products ===
C:\Users\kupit\AppData\Local\Temp\ipykernel_30600\952890231.py:278: PydanticDeprecatedSince20: The `dict` method is deprecated; use `model_dump` instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
  return listing_obj.dict()  # standardized format

=== Demo Summary ===
✅ No invalid products found, all products are valid
✅ 5 valid products processed successfully and JSON listings generated
```

JSON file: valid_product_listings.json
⚠️ Any API or JSON parsing errors were printed above